

FRONT-END WEB DEVELOPMENT

Dr Martin Kenirons

Martin.kenirons@atu.ie

Room 344A

Department of Computer Science & Applied Physics

1

MISCELLANEOUS

GENERIC

- *Generics allow a developer to create a component that can work over a variety of types rather than a single one.*

- ```
function identity(arg: number): number {
 return arg;
}
```

- Or, we could describe the identity function using the any type:

- ```
function identity(arg: any): any {  
    return arg;  
}
```

GENERIC

- *Generics allow a developer to create a component that can work over a variety of types rather than a single one.*

```
function identity(arg: number): number {  
    return arg;  
}
```

- Or, we could describe the identity function using the any type:

```
function identity(arg: any): any {  
    return arg;  
}
```

GENERIC

- use a type variable, a special kind of variable that works on types rather than values.

```
function identity<T>(arg: T): T {  
    return arg;  
}
```

ARROW FUNCTIONS

- More Concise Code.
- Simplifies function scoping and the 'this' keyword.
- Take a JavaScript Function and rewrite it

```
1  //JavaScript Function
2  function addition(x,y){
3      |   return x + y;
4      |
5      }
6
7  //Rewritten as an arrow function
8  let arrFuncRes = (x, y) => { return x + y };
9
10 console.log(arrFuncRes(2,3));
```

THIS

- The `this` keyword in JavaScript (and thus TypeScript) behaves differently than it does in many other languages. This can be very surprising, especially for users of other languages that have certain intuitions about how this should work.

```
class Foo {  
  x = 3;  
  print() {  
    console.log('x is ' + this.x);  
  }  
}  
  
var f = new Foo();  
f.print(); // Prints 'x is 3' as expected  
  
// Use the class method in an object literal  
var z = { x: 10, p: f.print };  
z.p(); // Prints 'x is 10'  
  
var p = z.p;  
p(); // Prints 'x is undefined'
```


ARRAYS

- 1d Array
- 2d Array
- https://www.tutorialspoint.com/typescript/typescript_multi_dimensional_arrays.htm

```
1 let multiArr: number [][] = [[1,2,3],[4,5,6],[7,8,9,10]];
2
3 console.log("The second element of the array within the first element of the main array is: "+multiArr[0][1]); // returns 2
4 console.log("The first element of the array within the second element of the main array is: "+multiArr[1][0]); // returns 4
5 console.log("The fourth element of the array within the third element of the main array is: "+multiArr[2][3]); // returns 10
6
7
```



ARRAYS

```
let multiArr: number [][] =  
[[15,23,13],[34,55,6],[87,38,19,110]];
```

```
console.log("What is : "+multiArr[0][2]);  
console.log("What is: "+multiArr[2][0]);  
console.log("What is: "+multiArr[2][3]);
```



MAP FUNCTION

- Transform arrays
- A function is applied to each element within the array.
- After the function is applied to each of the elements of the array, each of those elements is added to a new array.
- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/map

```
1
2 let singles: number[] = [1, 2, 3];
3 let doubles: number[] = singles.map(function(el){
4     return el * 2; }
5 );
6
7 console.log("Singles array: "+singles);
8 console.log("Doubles array: "+doubles);
```



MAP FUNCTION

- Transform arrays
- A function is applied to each element within the array.
- After the function is applied to each of the elements of the array, each of those elements is added to a new array.
- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/map

```
1
2 let singles: number[] = [1, 2, 3];
3 let doubles: number[] = singles.map(function(el){
4
5     return el * 2; }
6
7 );
8
9 console.log("Singles array: "+singles);
10 console.log("Doubles array: "+doubles);
```

```
Singles array: 1,2,3
Doubles array: 2,4,6
```



SWITCH STATEMENTS

- There can be any number of case statements within a switch.
- The case statements can include only constants. It cannot be a variable or an expression.
- The data type of the variable expression and the constant expression must match.
- Unless you put a break after each block of code, execution flows into the next block.
- The case expression must be unique.
- The default block is optional.

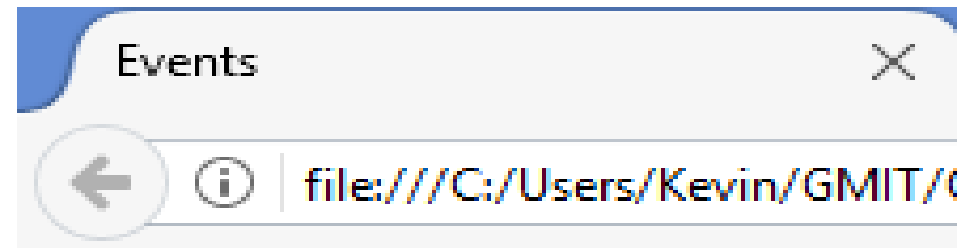
```
let theNumber:number = 1;
switch(theNumber) {
  case 1: {
    console.log("The number is 1.");
    break; //exits switch
  }
  case 2: {
    console.log("The number is 2.");
    break; //exits switch
  }
  case 3: {
    console.log("The number is 3.");
    break; //exits switch
  }
  default: {
    console.log("Invalid number. It must be 1, 2, or 3.");
    break; //optional; exits switch anyway
  }
}
```



DOM EXAMPLE

- https://www.w3schools.com/js/js_htmlDOM.asp
- <https://www.w3.org/DOM/>
- HTML Page / Web Page

```
1 <html>
2 <header>
3   <title>Events</title>
4 </header>
5 <body>
6   <button id="b1">Test</button>
7
8   <p id="p1">Original Test</p>
9   <script src="events.js"></script>
10 </body>
11 </html>
```



Test

Original Text



DOM EXAMPLE

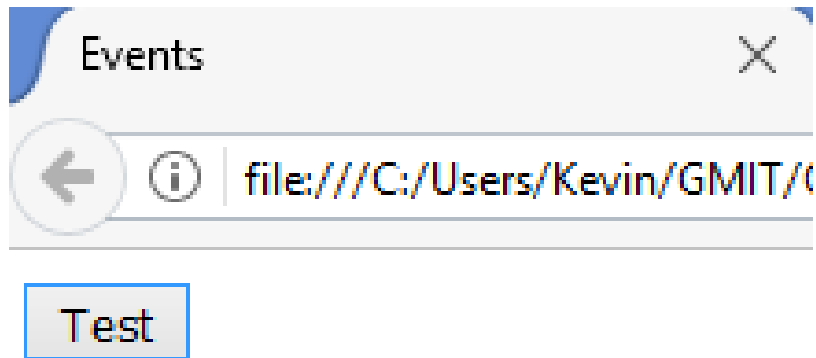
- Typescript

```
1  class Messages {
2      display_message(){
3          alert("Hello to you from the browser!!");
4      }
5      change_text(){
6          let el = document.getElementById("p1");
7          el.innerHTML = "Updated Text";
8          el.style.color="red";
9          el.style.fontSize = "14pt";
10     }
11 }
12
13
14 let messages = new Messages();
15
16 document.getElementById("b1").addEventListener("click", function(){
17     messages.change_text();
18     messages.display_message();
19 });
20
```

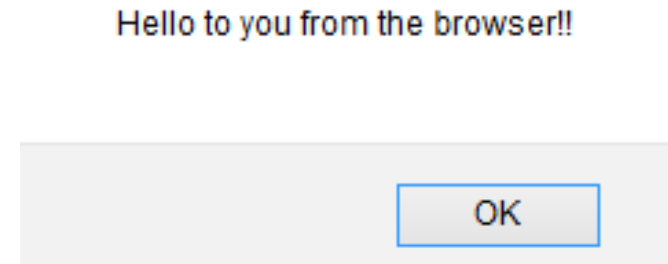


DOM EXAMPLE

- Web page after button click.



Updated Text



TEMPLATE STRINGS

- Angular Template Engine
- Template Example

```
<div>
  <h2>TypeScript Essentials</h2>
  <p>
    A good read!
  </p>
</div>
```

- Inline Example:

```
let template = "<div>\n  <h2>TypeScript Essentials</h2>\n  <p>\n    A good read!\n  </p>\n</div>";
```

- Backticks (`) Example:

```
let template = `
<div>
  <h2>TypeScript Essentials</h2>
  <p>
    A good read!
  </p>
</div>
`;
```



QUESTIONS